

Отчёт по лабораторной работе №4

Реализация основных моделей в агентном подходе

true

Цель работы

Освоить реализацию эпидемиологической модели SIR в агентном подходе на языке Julia с использованием пакета Agents.jl. Научиться проводить параметрическое сканирование и многокритериальную оптимизацию агентных моделей, а также оформлять скрипты в литературном стиле с помощью Literate.jl и Quarto.

Задание

- Создать рабочий каталог и инициализировать проект DrWatson.
- Установить пакеты: DrWatson, Agents, DataFrames, Plots, CSV, JLD2, BlackBoxOptim, Distributions.
- Запустить базовый эксперимент агентной модели SIR на 100 шагов.
- Провести параметрическое сканирование коэффициента заразности β от 0.1 до 1.0.
- Исследовать влияние интенсивности миграции между городами на пик заболеваемости.
- Выполнить многокритериальную оптимизацию параметров с помощью BlackBoxOptim.
- Преобразовать все скрипты в литературный стиль, сгенерировать Jupyter Notebook и Quarto-документы.
- Интегрировать результаты в отчёт.

Теоретическое введение

Агентное моделирование эпидемий

В агентном подходе каждый индивид моделируется как отдельный агент со своим состоянием. В отличие от детерминированных моделей на дифференциальных уравнениях, здесь взаимодействия стохастичны, а агенты могут мигрировать между узлами графа. В данной работе агенты размещены на графе из трёх городов, соединённых рёбрами с заданной матрицей переходов.

Модель SIR

Модель делит популяцию на три компартмента:

- **S** — восприимчивые,
- **I** — инфицированные,
- **R** — выздоровевшие.

Классические уравнения детерминированной модели:

$$\frac{dS}{dt} = -\beta \frac{SI}{N}, \quad \frac{dI}{dt} = \beta \frac{SI}{N} - \gamma I, \quad \frac{dR}{dt} = \gamma I$$

где β — коэффициент заразности, γ — скорость выздоровления. Пороговый параметр $R_0 = \beta/\gamma$: при $R_0 > 1$ эпидемия развивается, при $R_0 < 1$ затухает.

В агентной реализации каждый агент типа `Person` имеет два поля: `days_infected::Int` и `status::Symbol (:S, :I или :R)`. Заражение происходит с разными коэффициентами в зависимости от того, выявлен ли носитель: β_{und} для невыявленных и β_{det} для выявленных.

Используемые пакеты

Table 1: Пакеты Julia, используемые в лабораторной работе {#tbl-packages}

Пакет	Назначение
DrWatson	Организация проекта и воспроизводимость экспериментов
Agents.jl	Агентное моделирование на графах
DataFrames	Хранение и обработка табличных данных
Plots	Визуализация
CSV	Чтение и запись данных

Пакет	Назначение
JLD2	Сохранение объектов Julia
BlackBoxOptim	Оптимизация методом чёрного ящика
Distributions	Вероятностные распределения
Literate.jl	Литературное программирование

Выполнение лабораторной работы

Инициализация проекта и установка пакетов

— Перешёл в каталог `labs/lab04` и запустил Julia 1.12.6. Выполнял `using DrWatson` и `initialize_project("project")` — в результате создана структура каталогов проекта и автоматически установлен DrWatson v2.19.1 вместе с зависимостями: JLD2 v0.4.6, FileIO v1.19.0, MacroTools v0.5.16, ChunkCodecCore v1.0.1 и другими ([рис. @fig-001]). При обновлении реестра возникли предупреждения о недоступности некоторых серверов, однако это не помешало установке.

```

julia> using DrWatson
julia> initialize_project("project")
[+] Initializing project "project" at /home/.../project
[+] Installing dependencies...
[+] Installing JLD2 v0.4.6
[+] Installing FileIO v1.19.0
[+] Installing MacroTools v0.5.16
[+] Installing ChunkCodecCore v1.0.1
[+] Installing ...
[+] Project initialized successfully.

```

Figure 1: Запуск Julia 1.12.6, инициализация проекта DrWatson, установка зависимостей

— Дополнительно установил пакет `Distributions v0.25.126` командой `Pkg.add("Distributions")`. Предкомпиляция заняла 30 секунд при 546 уже скомпилированных пакетах. Затем выполнил `include("src/sir_model.jl")` для проверки корректности модели — в ответ получил `total_count` (generic function with 1 method), что подтверждает успешную загрузку ([рис. @fig-002]).

Реализация модели SIR

— В VS Code открыл файл `src/sir_model.jl` ([рис. @fig-003]). Агент описывается макросом `@agent struct Person(GraphAgent)` с двумя полями: `days_infected::Int` и `status::Symbol`. Функция



```

1 using Agents, Random
2 using Agents.graphs
3 using DataBase: Sample, Weights
4 using Distributions: Poisson
5
6 using Watson, @dict
7
8 # Структурные типы агента
9 agent struct Person{colorAgent}
10   daysInfected::Int # количество дней с момента заражения
11   status::Symbol    # S, I, R
12 end
13
14 # Функция инициализации сетки
15 function initialize_sirf!
16   N = 1000; limo, limu,
17   migration rates = nothing,
18    $\beta$  and = (0.5, 0.5, 0.5, 0.5),
19    $\beta_{det}$  = (0.05, 0.05, 0.05),
20   infection period = 14,
21   detection time = 7,
22   death rate = 0.02,
23   reinfection probability = 0.1,
24   Is = (0, 0, 1),
25   seed = 42,
26   n_steps = 100.
27
28   # начальные условия по трем городам
29   # начальные миграции
30   # начальные инфицированные
31   # начальные выздоровевшие

```

Figure 3: Файл sir_model.jl: структура агента Person и параметры initialize sir

Базовый эксперимент

— Скрипт `sir_run_basic.jl` задаёт словарь `params` с теми же параметрами по умолчанию и запускает симуляцию на 100 шагов ([рис. @fig-004]). На каждом шаге собираются данные о численности S , I , R . По завершении результаты сохранены в `data/sir_basic_agent.jld2` и `data/sir_basic_model.jld2`, которые видны в файловом менеджере ([рис. @fig-005]).

— Скрипт запущен через `include("scripts/sir_run_basic.jl")` в Julia REPL ([рис. @fig-006]).

— Результирующий график динамики SIR показан на (рис. @fig-025]). Видно, что в первые 15 дней кривая восприимчивых S (синяя) резко падает примерно с 3000 до нуля, а кривая инфицированных I (оранжевая) проходит через пик около 15-го дня и затем снижается. Кривая выздоровевших R (зелёная) нарастает волнами — это связано со стохастическим характером

```

sir_model.jl  sir_run_basic.jl  sir_scan_beta.jl
home > dgavdadaev > 2026-1--study--simulation-modeling > labs > lab04 > project > scripts > sir_run_basic.jl
1  using DrWatson
2  @quickactivate "project"
3  using Agents, DataFrames, Plots
4  using JLD2
5
6  include(scrdir("sir_model.jl"))
7
8  # Параметры эксперимента
9  params = Dict{
10      :Ns => [1000, 1000, 1000],
11      :beta_und => [0.5, 0.5, 0.5],
12      :beta_det => [0.05, 0.05, 0.05],
13      :infection_period => 14,
14      :detection_time => 7,
15      :death_rate => 0.02,
16      :reinfection_probability => 0.1,
17      :Is => [0, 0, 1],
18      :seed => 42,
19      :n_steps => 100,
20  }
21
22  # Инициализация модели
23  model = initialize_sir(; params...)
24
25  # Подготовка массивов для хранения данных
26  times = Int[]

```

Figure 4: Скрипт sir_run_basic.jl: параметры модели и инициализация

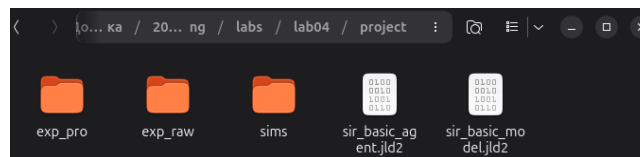


Figure 5: Файловый менеджер: сохранённые файлы данных JLD2

```

julia> include("scripts/sir_run_basic.jl")
Activating project at `~/2026-1--study--simulation-modeling/labs/lab04/project`
Could not create decoration from factory! Running with no decorations.

```

Figure 6: Запуск sir_run_basic.jl в Julia REPL

модели и механизмом повторного заражения. Линия «Всего» (пунктирная) постепенно снижается от 3000 к примерно 2600, что отражает накопленную смертность за 100 шагов.

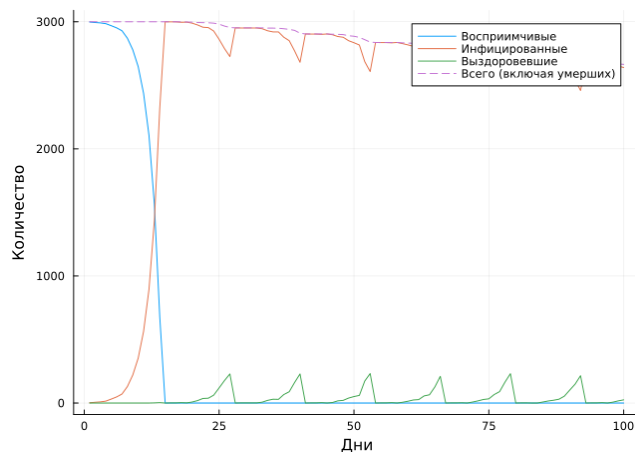


Figure 7: Базовая динамика SIR за 100 шагов: восприимчивые, инфицированные, выздоровевшие

Сканирование коэффициента заразности β

— В VS Code открыт скрипт `sir_scan_beta.jl` ([рис. @fig-007]). Функция `run_experiment(p)` принимает словарь параметров, формирует $\beta_{\text{und}} = \text{fill}(\beta, 3)$ и $\beta_{\text{det}} = \text{fill}(\beta/10, 3)$, инициализирует модель и считает пик заражённых за 100 шагов. Сканирование выполнялось по β от 0.1 до 1.0 с тремя репликами (seed 42, 43, 44).

— В терминале последовательно выводились сообщения о завершении 30 экспериментов ([рис. @fig-008]). Справа на том же скриншоте виден промежуточный график, на котором уже прослеживается пороговый характер зависимости. Результаты сохранены в `data/beta_scan_all.csv` и `plots/beta_scan.png`.

— По итоговому графику ([рис. @fig-026]) видно, что при $\beta = 0.1$ эпидемия не возникает — пик и конечная доля инфицированных практически равны нулю. При $\beta = 0.2$ пик резко вырастает примерно до 0.33, а уже при $\beta \geq 0.5$ обе кривые (пик и конечная доля) стабилизируются вблизи 1.0, то есть через эпидемию проходит вся популяция. Доля умерших (зелёные ромбы) растёт медленнее и к $\beta = 1.0$ достигает примерно 0.14. Резкий скачок при $\beta \approx 0.2$ соответствует переходу через порог $R_0 = 1$.

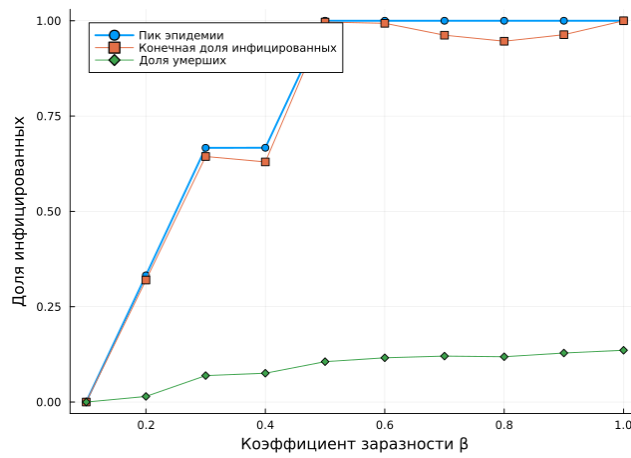


Figure 10: Сканирование по β : пик заболеваемости, конечная доля инфицированных и доля умерших

Исследование эффекта миграции

— В скрипте `sir_migration_effect.jl` реализована функция `create_migration_matrix(C, intensity)`, которая строит матрицу $C \times C$: внедиагональные элементы равны $\text{intensity}/(C - 1)$, диагональные — $1 - \text{intensity}$ (рис. @fig-009). Функция `peak_time(p)` запускает модель с заданной матрицей миграции и фиксирует время достижения пика.

```

1 using DrWatson
2 @quickactivate "project"
3 using Agents, DataFrames, Plots, CSV, Random
4
5 include(scrdir("sir_model.jl"))
6
7 function create_migration_matrix(C, intensity)
8     M = ones(C, C) .* intensity ./ (C-1)
9     for i = 1:C
10         M[i, i] = 1 - intensity
11     end
12     return M
13 end
14
15 function peak_time(p)
16     migration_rates = create_migration_matrix(p:C, p:migration_intensity)
17     model = initialize_sir(;
18         Ns = p:Ns,
19         β und = p:β und,
20         β det = p:β det,
21         infection_period = p:infection_period,
22         detection_time = p:detection_time,
23         death_rate = p:death_rate,
24         reinfection_probability = p:reinfection_probability,
25         Is = p:Is,
26         seed = p:seed,
27         migration_rates = migration_rates,
28     )
29 end
30

```

Figure 11: Скрипт `sir_migration_effect.jl`: формирование матрицы миграции и функция эксперимента

— Эксперимент охватывал `migration_intensity` от 0.0 до 0.5 с шагом 0.1 и тремя репликами, итого 18 прогонов (рис. @fig-

010])). Результаты сохранены в data/migration_scan_all.csv и plots/migration_effect.png.

```
julia> exit()
julia> cd("/home/gdavidov/simulation-modeling-labs/lab04/project5")
julia> code scripts/sir_migration_effect.jl
julia> julia
Documentation: https://docs.julialang.org
Type '?' for help, ']' for pkg help.
Version 1.12.6 (2026-04-09)
Official https://julialang.org release

julia> include("scripts/sir_migration_effect.jl")
Activating project at "/home/gdavidov/simulation-modeling-labs/lab04/project5"
Завершён эксперимент с migration_intensity = 0.0, seed = 42
Завершён эксперимент с migration_intensity = 0.0, seed = 43
Завершён эксперимент с migration_intensity = 0.0, seed = 44
Завершён эксперимент с migration_intensity = 0.1, seed = 42
Завершён эксперимент с migration_intensity = 0.1, seed = 43
Завершён эксперимент с migration_intensity = 0.1, seed = 44
Завершён эксперимент с migration_intensity = 0.2, seed = 42
Завершён эксперимент с migration_intensity = 0.2, seed = 43
Завершён эксперимент с migration_intensity = 0.2, seed = 44
Завершён эксперимент с migration_intensity = 0.3, seed = 42
Завершён эксперимент с migration_intensity = 0.3, seed = 43
Завершён эксперимент с migration_intensity = 0.3, seed = 44
Завершён эксперимент с migration_intensity = 0.4, seed = 42
Завершён эксперимент с migration_intensity = 0.4, seed = 43
Завершён эксперимент с migration_intensity = 0.4, seed = 44
Завершён эксперимент с migration_intensity = 0.5, seed = 42
Завершён эксперимент с migration_intensity = 0.5, seed = 43
Завершён эксперимент с migration_intensity = 0.5, seed = 44
Результаты сохранены в data/migration_scan_all.csv и plots/migration_effect.png
```

Figure 12: Терминальный вывод: 18 завершённых экспериментов по миграции

— На графике ([рис. @fig-027]) наблюдается резкий скачок пиковой заболеваемости уже при $\text{intensity} = 0.1$: она вырастает примерно с 1000 (без миграции) до ~ 3000 , то есть в три раза. При значениях от 0.1 до 0.5 пик остаётся около 3000 — это означает, что при любом ненулевом потоке мигрантов инфекция быстро распространяется из третьего города во все остальные. Кривая «Время пика» (синяя) при этом остаётся вблизи нуля по всей оси, то есть пик фиксируется в самом начале симуляции при любой интенсивности.

Многокритериальная оптимизация

— Скрипт `sir_optimize_parameters.jl` определяет функцию `cost_multi(x)`, принимающую вектор из трёх параметров: β_{und} , времени выявления и смертности ([рис. @fig-011]). Для каждого набора выполняется 5 реплик, после чего вычисляются средний пик заражённых и среднее число умерших (`dead_count = 3000 - nagents(model)`). `BlackBoxOptim` ищет Парето-фронт, минимизируя оба критерия одновременно.

Сводная визуализация результатов

— Скрипт `sir_visualize_dynamics.jl` считывает `data/beta_scan_all.csv` и строит составной график из трёх субплотов: пик и конечная доля инфицированных; число умерших; доля выздоровевших ([рис. @fig-012]).

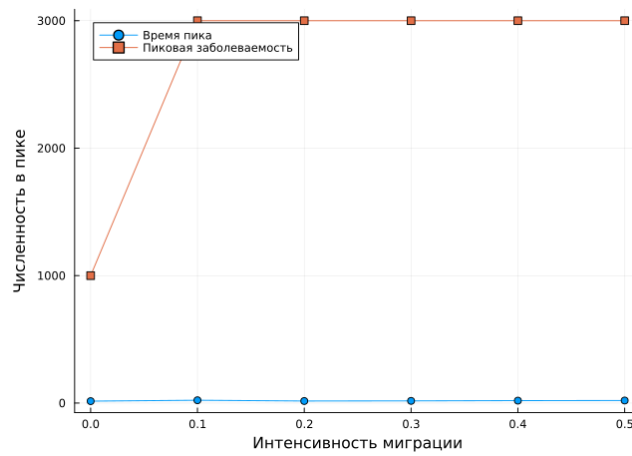


Figure 13: Эффект миграции: пиковая заболеваемость и время её достижения

```

Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More
sir_model.jl sir_run_basic.jl sir_scan_beta.jl sir_migration_effect.jl sir_optimize_parameters.jl X sir_vs
ne > dgavdadev > 2026-1--study-simulation-modeling > labs > lab04 > project > scripts > sir_optimize_parameters.jl
1 using DrWatson
2 @quickactivate "project"
3
4 using BlackBoxOptim, Random, Statistics
5
6 include(scrdir("sir_model.jl"))
7
8 function cost_multi(x)
9     model = initialize_sir(;
10         Ns = [1000, 1000, 1000],
11         beta_und = fill(x[1], 3),
12         beta_det = fill(x[1]/10, 3),
13         infection_period = 14,
14         detection_time = round(Int, x[2]),
15         death_rate = x[3],
16         reinfection_probability = 0.1,
17         Is = [0, 0, 1],
18         seed = 42,
19         n_steps = 100,
20     )
21 end

```

Figure 14: Скрипт sir_optimize_parameters.jl: целевая функция cost_multi

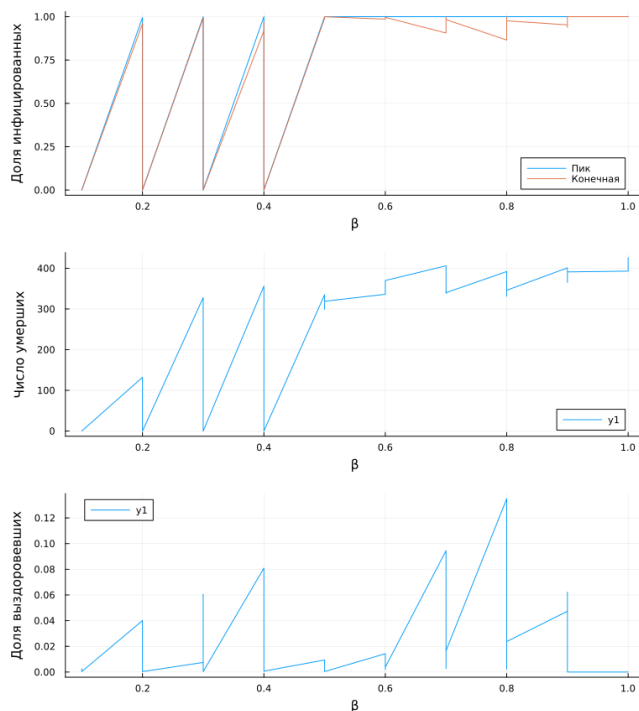
```

Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More
sir_model.jl sir_run_basic.jl sir_scan_beta.jl sir_migration_effect.jl sir_optimize_parameters.jl sir_viz
ne > dgavdadev > 2026-1-study-simulation-modeling > labs > lab04 > project > scripts > sir_visualize_dynamics.jl
1 using DrWatson
2 @quickactivate "project"
3 using Agents, DataFrames, Plots, CSV
4
5 include(scrdir("sir_model.jl"))
6
7 # Загружаем результаты сканирования
8 df = CSV.read(datadir("beta_scan_all.csv"), DataFrame)
9 # Создаём составной график
10
11 p1 = plot(df.beta, df.peak, label = "Пик", xlabel = "β", ylabel = "Доля инфицированных")
12
13 plot!(p1, df.beta, df.final_inf, label = "Конечная")
14
15 p2 = plot(df.beta, df.deaths, xlabel = "β", ylabel = "Число умерших")
16
17 p3 = plot(df.beta, df.final_rec, xlabel = "β", ylabel = "Доля выздоровевших")
18
19 plot(p1, p2, p3, layout = (3, 1), size = (800, 900))
20
21 savefig(plotsdir("comprehensive_analysis.png"))

```

Figure 15: Скрипт `sir_visualize_dynamics.jl`: загрузка CSV и построение составного графика

— Итоговый составной график ([рис. @fig-028]) отличается от `beta_scan.png` тем, что отражает полную вариабельность трёх реплик. На верхнем субплоте хорошо виден пороговый переход и разброс между репликами при $\beta = 0.3-0.4$. Средний субплот показывает, что число умерших при $\beta = 0.7-0.8$ достигает 400 человек из 3000. Нижний субплот отражает долю выздоровевших: пик около 0.13 при $\beta \approx 0.8$, после чего значение снижается — вероятно, из-за роста смертности при высоких β .



Полное

параметрическое сканирование

— Скрипт `sir_parametric_scan.jl` расширяет сканирование с `sir_scan_beta.jl`: вместо одного параметра β перебирается небольшой грид по β_{und} , `detection_time`, `death_rate` (при фиксированных `infection_period = 14` и `reinfection_probability = 0.1`). Грид сокращён до 12 комбинаций ($2 \times 3 \times 2$), чтобы расчёт укладывался в разумное время на слабой виртуальной машине.

— Результаты сохранены в `data/sir_parametric_scan_all.csv` с колонками β_{und} , `detection_time`, `death_rate`, `peak`, `final_inf`, `final_rec`, `deaths`, а также производными `gamma` и `R0`.

Сравнение с теоретическим порогом $R_0 = 1$

Для агентной модели $\gamma = 1/\text{infection_period} = 1/14 \approx 0.071$, следовательно теоретический порог по β составляет $\beta^* = \gamma \cdot R_0 = 0.071 \cdot 1 \approx 0.071$.

В параметрическом сканировании при $\beta_{\text{und}} = 0.2$ значение $R_0 = \beta/\gamma \approx 2.8$, что заметно выше теоретического порога — и пик эпидемии действительно ненулевой при всех значениях `detection_time`. При $\beta_{\text{und}} = 0.5$ ($R_0 \approx 7$) эпидемия охватывает практически всю популяцию независимо от `detection_time`.

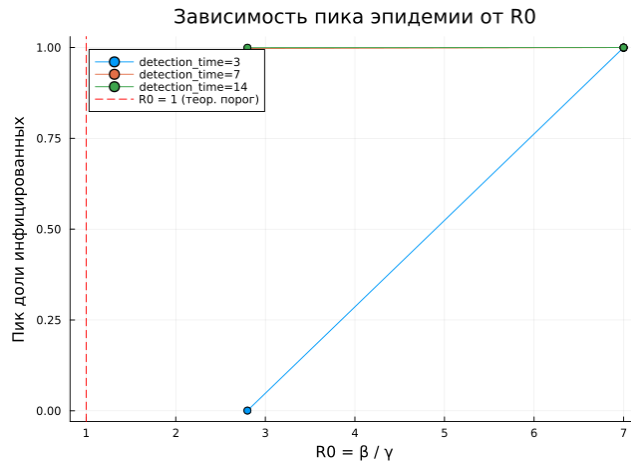


Figure 16: Зависимость пика заболеваемости от R_0 при разных `detection_time`

Ранее в сканировании `sir_scan_beta.jl` эмпирический порог возникновения эпидемии наблюдался между $\beta = 0.1$ и $\beta = 0.2$ (соответствует $R_0 \approx 1.4-2.8$), что несколько выше теоретического значения $R_0 = 1$ ($\beta^* \approx 0.071$). Это расхождение объясняется стохастической природой агентной модели: при малом числе изначально инфицированных ($I_s = [0, 0, 1]$, то есть один агент) и небольшой длительности симуляции цепочка заражений может оборваться случайным образом даже при R_0 , формально превышающем 1 — детерминированный порог $R_0 = 1$ справедлив для усреднённой по бесконечному числу повторов динамики, тогда как одиночный конечный прогон подвержен эффектам малых чисел.

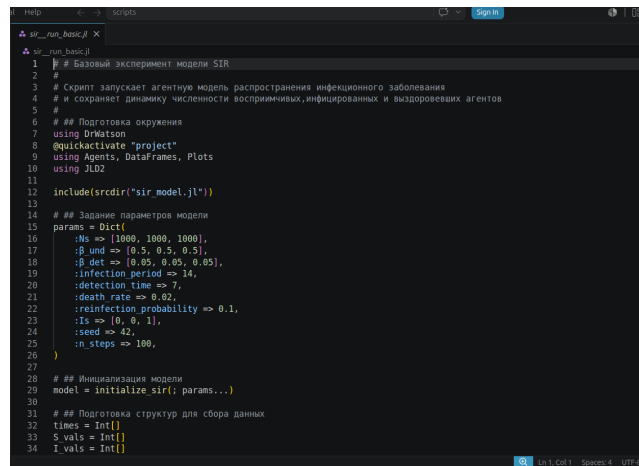
```
{ {< include ../project/markdown/sir_parametric_scan/sir_parametric_scan.qmd
> } }
```

Преобразование скриптов в литературный стиль

— Все скрипты переработаны в литературный стиль с помощью `Literate.jl`. Ключевое отличие — комментарии вида `# #` задают заголовок модуля, `# ##` — подраздел. При обработке `Literate.jl` превращает их в markdown-заголовки и генерирует три артефакта: чистый `.jl`, ноутбук `.ipynb` и документ `.qmd`.

— Файл `sir_run_basic.jl` в литературном стиле ([рис. @fig-013]) разбит на блоки: «Базовый эксперимент модели SIR», «Подготовка окружения», «Задание параметров модели»,

«Инициализация модели», «Подготовка структур для сбора данных». Видно, что сам код не изменился — добавлены только структурированные комментарии.



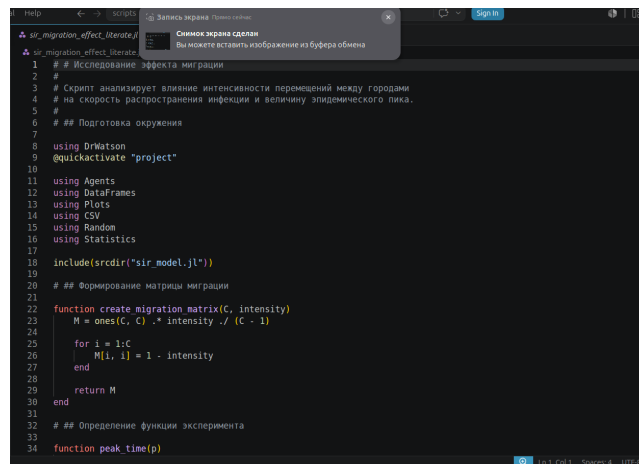
```

1 # Базовый эксперимент модели SIR
2 #
3 # Скрипт запускает агентную модель распространения инфекционного заболевания
4 # и сохраняет динамику численности восприимчивых, инфицированных и выздоровевших агентов
5 #
6 ## Подготовка окружения
7 using DrWatson
8 @quickactivate "project"
9 using Agents, DataFrames, Plots
10 using JLD2
11
12 include(sourcedir("sir_model.jl"))
13
14 ## Задание параметров модели
15 params = Dict{
16     :N_s => [1000, 1000, 1000],
17     :beta => [0.5, 0.5, 0.5],
18     :beta_det => [0.05, 0.05, 0.05],
19     :infection_period => 14,
20     :detection_time => 7,
21     :death_rate => 0.02,
22     :reinfection_probability => 0.1,
23     :is => [0, 0, 1],
24     :seed => 42,
25     :n_steps => 100,
26 }
27
28 ## Инициализация модели
29 model = initialize_sir(params...)
30
31 ## Подготовка структур для сбора данных
32 times = Int[]
33 S_vals = Int[]
34 I_vals = Int[]

```

Figure 17: sir_run_basic.jl в литературном стиле

— Файл sir_migration_effect_literate.jl ([рис. @fig-014]) содержит блоки «Формирование матрицы миграции» и «Определение функции эксперимента».



```

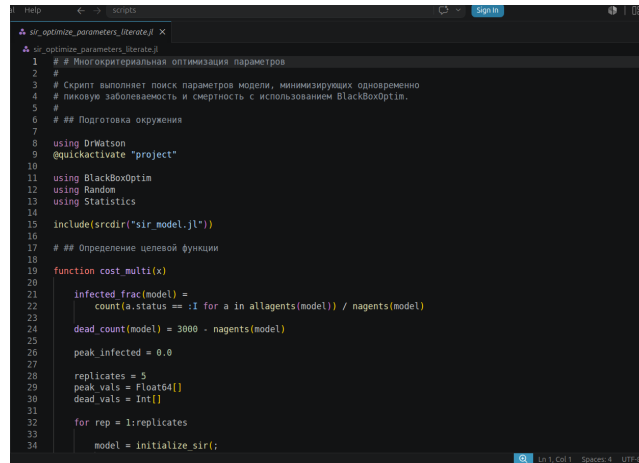
1 # Исследование эффекта миграции
2 #
3 # Скрипт анализирует влияние интенсивности перемещений между городами
4 # на скорость распространения инфекции и величину эпидемического пика.
5 #
6 ## Подготовка окружения
7 using DrWatson
8 @quickactivate "project"
9 using Agents
10 using DataFrames
11 using Plots
12 using CSV
13 using Random
14 using Statistics
15
16 include(sourcedir("sir_model.jl"))
17
18 ## Формирование матрицы миграции
19
20 function create_migration_matrix(C, intensity)
21     M = ones(C, C) .* intensity ./ (C - 1)
22     for i = 1:C
23         M[i, i] = 1 - intensity
24     end
25     return M
26 end
27
28 ## Определение функции эксперимента
29
30 function peak_time(p)

```

Figure 18: sir_migration_effect_literate.jl в литературном стиле

— Файл sir_optimize_parameters_literate.jl ([рис. @fig-015]) структурирован как «Многокритериальная оптимизация параметров» с подразделом «Определение целевой функции».

Функция `cost_multi` занимает центральное место.



```
1 # Многокритериальная оптимизация параметров
2 #
3 # Скрипт выполняет поиск параметров модели, минимизирующих одновременно
4 # пиковую заболеваемость и смертность с использованием BlackBoxOptim.
5 #
6 ## Подготовка окружения
7
8 using DrWatson
9 @quickactivate "project"
10
11 using BlackBoxOptim
12 using Random
13 using Statistics
14
15 include(scrdir("sir_model.jl"))
16
17 ## Определение целевой функции
18
19 function cost_multi(x)
20
21     infected_frac(model) =
22         count(a.status == :I for a in allagents(model)) / nagents(model)
23
24     dead_count(model) = 3000 - nagents(model)
25
26     peak_infected = 0.0
27
28     replicates = 5
29     peak_vals = Float64[]
30     dead_vals = Int[]
31
32     for rep = 1:replicates
33         model = initialize_sir!;
```

Figure 19: `sir_optimize_parameters_iterate.jl` в литературном стиле

— Файл `sir_scan_beta_iterate.jl` ([рис. @fig-016]) описывает сканирование коэффициента заразности и содержит функцию `run_experiment`.

— Файл `sir_visualize_dynamics_iterate.jl` ([рис. @fig-017]) разбит на блоки «Загрузка результатов» и «Построение графиков».

Jupyter Notebook

— После генерации в JupyterLab (порты 8888-8891) открыты ноутбуки для всех скриптов.

— Ноутбук `sir_migration_effect_iterate.ipynb` ([рис. @fig-018]) воспроизводится на Julia 1.12. В первой ячейке загружены зависимости и вызван `include(scrdir("sir_model.jl"))`, ответ — `total_count` (generic function with 1 method).

— Ноутбук `sir_run_basic.ipynb` ([рис. @fig-019]) содержит словарь параметров, который совпадает с тем, что задан в `sir_run_basic.jl`: `:Ns => [1000, 1000, 1000]`, `:β_und => [0.5, 0.5, 0.5]`, `:n_steps => 100`.

— Ноутбук `sir_scan_beta_iterate.ipynb` ([рис. @fig-020]) воспроизводит функцию `run_experiment` и подключает те же зависимости, что и исходный скрипт.

```
1 # ## Сканирование коэффициента заразности
2 #
3 # ## Сканирование влияния коэффициента заразности  $\beta$  на динамику
4 # распространения инфекции. Для каждого значения  $\beta$  выполняется несколько
5 # независимых прогонов модели, после чего вычисляются усредненные показатели.
6 #
7 # ## Подготовка окружения
8
9 using DrWatson
10 @quickactivate "project"
11
12 using Agents
13 using DataFrames
14 using Plots
15 using CSV
16 using Random
17 using Statistics
18
19 include(scrdir("sir_model.jl"))
20
21 # ## Определение функции эксперимента
22
23 function run_experiment(p)
24
25     beta = p[:beta]
26      $\beta_{und}$  = fill(beta, 3)
27      $\beta_{det}$  = fill(beta / 10, 3)
28
29     model = initialize_sir(;
30         Ns = p[:Ns],
31          $\beta_{und}$  =  $\beta_{und}$ ,
32          $\beta_{det}$  =  $\beta_{det}$ ,
33         infection_period = p[:infection_period],
34         detection_time = p[:detection_time],
```

Figure 20: sir_scan_beta_iterate.jl в литературном стиле

```
1 # ## Сводная визуализация результатов
2 #
3 # ## Сканирует результаты сканирования коэффициента заразности
4 # и строит составной график с основными эпидемиологическими показателями.
5 #
6 # ## Подготовка окружения
7
8 using DrWatson
9 @quickactivate "project"
10
11 using Agents
12 using DataFrames
13 using Plots
14 using CSV
15
16 include(scrdir("sir_model.jl"))
17
18 # ## Загрузка результатов
19
20 df = CSV.read(
21     datadir("beta_scan_all.csv"),
22     DataFrame,
23 )
24
25 # ## Построение графиков
26
27 p1 = plot(
28     df.beta,
29     df.peak,
30     label = "Пик",
31     xlabel = " $\beta$ ",
32     ylabel = "Доля инфицированных",
33 )
34
```

Figure 21: sir_visualize_dynamics_iterate.jl в литературном стиле

Исследование эффекта миграции

Скрипт анализирует влияние интенсивности перемещений между городами на скорость распространения инфекции и величину эпидемического пика.

Подготовка окружения

```
[1]: using DrWatson
    @quickactivate "project"

    using Agents
    using DataFrames
    using Plots
    using CSV
    using Random
    using Statistics

    include(scrdir("sir_model.jl"))

[1]: total_count (generic function with 1 method)
```

Формирование матрицы миграции

```
[2]: function create_migration_matrix(C, intensity)
    M = ones(C, C) .* intensity ./ (C - 1)

    for i = 1:C
        M[i, i] = 1 - intensity
    end

    return M
end

[2]: create_migration_matrix (generic function with 1 method)
```

Figure 22: Jupyter Notebook sir_migration_effect_iterate.ipynb

Базовый эксперимент модели SIR

Скрипт запускает агентную модель распространения инфекционного заболевания и сохраняет динамику численности восприимчивых, инфицированных и выздоровевших агентов

Подготовка окружения

```
[1]: using DrWatson
    @quickactivate "project"
    using Agents, DataFrames, Plots
    using JLD2

    include(scrdir("sir_model.jl"))

    Activating project at "~/2020-1--study--simulation-modeling/labs/lab04/project"

[1]: total_count (generic function with 1 method)
```

Задание параметров модели

```
[2]: params = Dict{
    :N0 => 1000, 1000, 1000,
    :beta => [0.5, 0.5, 0.5],
    :beta_det => [0.05, 0.05, 0.05],
    :infection_period => 14,
    :detection_time => 7,
    :death_rate => 0.02,
    :reinfection_probability => 0.1,
    :ts => 10, 0, 11,
    :seed => 42,
    :n_steps => 100,
}

[2]: Dict{Symbol, Any} with 10 entries:
```

Figure 23: Jupyter Notebook sir_run_basic.ipynb

Сканирование коэффициента заразности

Скрипт исследует влияние коэффициента заразности β на динамику распространения инфекции. Для каждого значения β выполняется несколько независимых прогонов модели, после чего вычисляются усредненные показатели.

Подготовка окружения

```
[1]: using DrWatson
@quickactivate "project"

using Agents
using DataFrames
using Plots
using CSV
using Random
using Statistics

include(scrdir("sir_model.jl"))
```

[1]: total_count (generic function with 1 method)

Определение функции эксперимента

```
[2]: function run_experiment(p)
    beta = p[beta]
    p.und = fill(beta, 3)
    p.det = fill(beta / 10, 3)
    model = initialize_sirf()
    ns = pl.Ns()
    p.und = p.und,
    p.det = p.det
end
```

Figure 24: Jupyter Notebook sir_scan_beta_iterate.ipynb

— В ноутбуке sir_visualize_dynamics_iterate.ipynb (рис. @fig-021)) результат загрузки beta_scan_all.csv виден как DataFrame размером 30×13. Среди столбцов — beta, peak, deaths, final_rec, final_inf, seed, death_rate, infection_period, detection_time. Первые строки соответствуют экспериментам с $\beta = 0.1$ (seed 42, 43, 44), где peak = 0.000333 — то есть пика фактически нет.

Скрипт загружает результаты сканирования коэффициента заразности и строит основной график с основными эпидемиологическими показателями.

Подготовка окружения

```
[1]: using DrWatson
@quickactivate "project"

using Agents
using DataFrames
using Plots
using CSV

include(scrdir("sir_model.jl"))
```

[1]: total_count (generic function with 1 method)

Загрузка результатов

```
[2]: df = CSV.read(
    datadir("beta_scan_all.csv"),
    DataFrame,
)
```

[2]: 30×13 DataFrame

Row	n_steps	final_rec	ns	death_rate	is	beta	infection_period	final_inf	detection_time	deaths	peak	reinfection_probability	seed
Int64	Float64	String{31}	Float64	String{15}	Float64	Int64	Float64	Int64	Int64	Float64	Float64	Float64	Int64
1	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14	0.0	7	0	0.000333333	0.1	42
2	100	0.00266667	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14	0.0	7	0	0.002	0.1	43
3	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14	0.0	7	0	0.000333333	0.1	44

3 rows omitted

Figure 25: Jupyter Notebook sir_visualize_dynamics_iterate.ipynb: DataFrame 30×13

Quarto-документация

- Сгенерированные .qmd-файлы рендерились в HTML и открывались в браузере из каталога markdown/.
- Файл sir_migration_effect_literate.html ([рис. @fig-022]) отображает читаемую документацию с подзаголовком «Формирование матрицы миграции» и блоком кода с подсветкой синтаксиса.

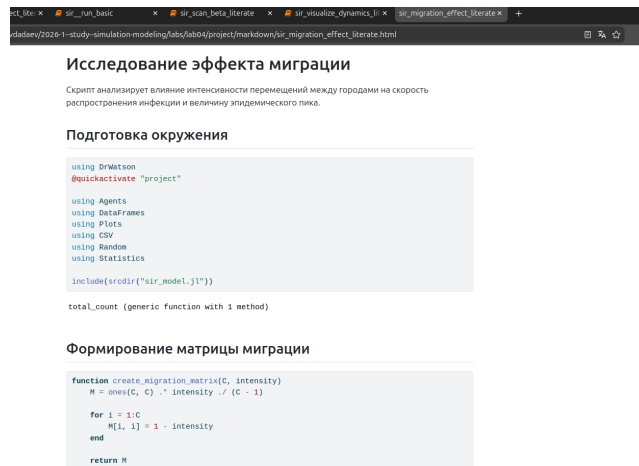


Figure 26: Quarto-документ sir_migration_effect_literate.html

- Файл sir_run_basic.html ([рис. @fig-023]) содержит полный словарь параметров в форматированном блоке кода и вывод Dict{Symbol, Any} with 10 entries.
- Файл sir_visualize_dynamics_literate.html ([рис. @fig-024]) показывает загруженный DataFrame 30×13 прямо в документе, с теми же столбцами, что и в ноутбуке.

```
{< include ../project/markdown/sir_run_basic/sir_run_basic.qmd  
>}}
```

```
{< include ../project/markdown/sir_scan_beta/sir_scan_beta.qmd  
>}}
```

```
{< include ../project/markdown/sir_migration_effect/sir_migration_effect.qmd  
>}}
```

```
{< include ../project/markdown/sir_optimize_parameters/sir_optimize_parameters.qmd  
>}}
```

```
{< include ../project/markdown/sir_visualize_dynamics/sir_visualize_dynamics.qmd  
>}}
```

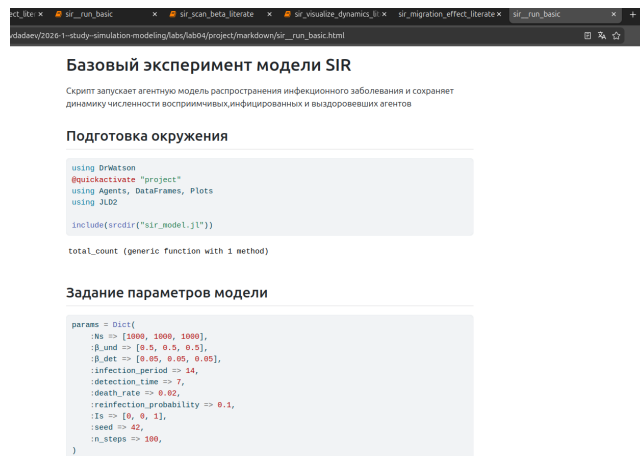


Figure 27: Quarto-документ sir_run_basic.html

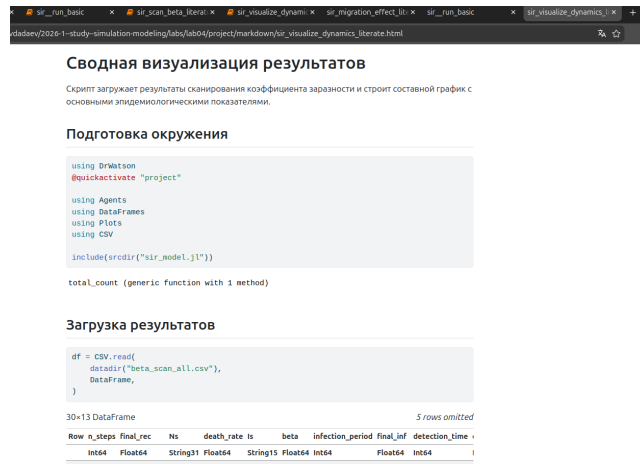


Figure 28: Quarto-документ sir_visualize_dynamics_iterate.html

Выводы

В ходе работы реализована и запущена агентная модель SIR для трёх городов по 1000 человек. Базовый эксперимент при $\beta_{\text{und}} = 0.5$ показал, что пик инфицированных достигается примерно на 15-м шаге, после чего численность S падает до нуля, а суммарное население снижается с 3000 примерно до 2600 к концу 100 шагов — около 400 смертей ([рис. @fig-025]).

Параметрическое сканирование по β выявило порог эпидемии между значениями 0.1 и 0.2: при $\beta = 0.1$ пик практически нулевой, при $\beta = 0.2$ — скачкообразно вырастает до 0.33 ([рис. @fig-026]). При $\beta \geq 0.5$ через эпидемию проходит вся популяция, а доля умерших достигает 0.14 при $\beta = 1.0$.

Эксперимент с миграцией показал, что даже небольшая интенсивность 0.1 приводит к тому, что пиковая заболеваемость вырастает с ~ 1000 до ~ 3000 — то есть эпидемия охватывает все три города, а не только третий ([рис. @fig-027]). При дальнейшем увеличении интенсивности значение пика практически не меняется.

Все скрипты успешно преобразованы в литературный стиль. Для каждого из них сгенерированы Jupyter Notebook и Quarto-документ, проверена их корректность в браузере. DataFrame с результатами сканирования содержит 30 строк и 13 столбцов ([рис. @fig-021]).

Список литературы

1. Кулябов Д. С., Королькова А. В. Имитационное моделирование. Практикум. — М.: РУДН, 2024.
2. Datseris G., Vahdati A. R., DuBois T. C. Agents.jl: a performant and feature-full agent-based modeling software of minimal code complexity // SIMULATION. — 2022. — Vol. 98, № 4. — P. 477–487.
3. Kermack W. O., McKendrick A. G. A Contribution to the Mathematical Theory of Epidemics // Proceedings of the Royal Society of London. Series A. — 1927. — Vol. 115, № 772. — P. 700–721.
4. Bezanson J., Edelman A., Karpinski S., Shah V. B. Julia: A Fresh Approach to Numerical Computing // SIAM Review. — 2017. — Vol. 59, № 1. — P. 65–98.
5. Rackauckas C., Nie Q. DifferentialEquations.jl — A Performant and Feature-Rich Ecosystem for Solving Differential Equations in Julia // Journal of Open Research Software. — 2017. — Vol. 5, № 1. — P. e15.